

Anhang A

Einführung in UNIX

A.1 Grundlagen

UNIX ist ein *time-sharing-Betriebssystem*. Dieses bedeutet, dass sich mehrere Benutzer an verschiedenen Terminals gleichzeitig die Ressourcen eines Rechners (Speicher, Prozessorzeit, Platten, Drucker, usw.) teilen können. Es ist aber auch möglich, dass ein einzelner Benutzer mehrere Prozesse gleichzeitig laufen lassen kann. Wenn mehrere Prozesse eines oder mehrerer Benutzer gleichzeitig laufen, muss das Betriebssystem die Prozessorzeit den einzelnen Prozessen abwechselnd nach einem Prioritätenschema zuweisen. Während die Prozessorzeit also für kurze Intervalle ausschließlich einem einzelnen Prozess zur Verfügung steht, werden andere Ressourcen (wie z.B. der Speicher) permanent einem Prozess zugeordnet. Ein Prozess belegt also auch dann Speicher, wenn zwar gestartet wurde, sich aber nicht in der Ausführung befindet.

Jeder Benutzer besitzt einen Benutzernamen und ein hoffentlich nur ihm und dem System bekanntes Passwort. Unter den Benutzern gibt es einen, der unbeschränkte Privilegien besitzt, mit dem Namen *root*. Der Benutzer *root* kann dem System einen neuen Benutzernamen mit Passwort mitteilen und somit einem neuen Benutzer den Zugang zum System ermöglichen. Genauso kann er den Namen eines Benutzer aus dem System entfernen und somit diesem die Rechenberechtigung entziehen.

A.2 Ein- und Ausloggen, Passwort

Jeder Benutzer muss sich vor jeder Sitzung am Terminal einer Zulassungsprozedur zum Rechnen unterwerfen, dem *login*-Verfahren. Wie diese Prozedur im einzelnen abläuft, hängt sehr stark von der jeweiligen Umgebung ab.

Im wesentlichen werden Benutzername und Passwort abgefragt. Das kann in einer Konsole oder einem grafischen login-Fenster geschehen. Heutzutage werden fast alle Systeme auch eine grafische Benutzeroberfläche anbieten. Dann kann man nach erfolgreichem login über die Oberfläche ein Konsolenfenster öffnen. Der Rest dieses Abschnittes wird weiter auf die Nutzung der Konsole eingehen. Von dort aus kann man alle wichtigen Operationen ausführen. Eine Behandlung von grafischen Benutzeroberflächen ist angesichts derer Vielfalt hier nicht möglich.

Hat man nun Zugriff auf eine Konsole, erscheint dort dann der Textcursor hinter dem Prompt des Systems, das die Form

```
rechnername>
```

hat. Das Beenden einer Sitzung erfolgt durch die Eingabe des Kommandos

```
exit ↵
```

oder noch einfacher durch Eingabe von `C-d` im Terminalfenster. Das Zeichen `↵` bedeutet Drücken der `ENTER`-Taste; `C-d` bedeutet, dass man die Taste `Ctrl` gedrückt hält und dazu die Taste `d` anschlägt.

A.3 Hilfen

Die Beschreibung von beliebigen Kommandos kann man sich auf dem Bildschirm mit dem Befehl

```
man kommandoname ↵
```

(*manual*) ansehen. Man erhält dann die entsprechenden Seiten des UNIX-Manuals aufgelistet. So erhält man die Erläuterung des Kommandos `passwd` durch

```
man passwd ↵
```

und die Erläuterung des Kommandos `man` selbst durch

```
man man ↵
```

Die UNIX-Manuals sind in verschiedene Abschnitte aufgeteilt. `man` liefert das erste Ergebnis, egal in welchem Bereich es gefunden wurde. Möchte man speziell z.B. die Dokumentation zu einer Bibliotheksfunktion erhalten, so muss man den Abschnitt mit angeben. Dies geschieht durch eine zusätzliche Zahl vor dem eigentlichen Begriff. Mit

```
man 3 printf ↵
```

erhält man so die Dokumentation zu `printf` aus der C-Standardbibliothek.

A.4 Das Dateisystem

Der gesamte Massenspeicher, über den ein UNIX-System verfügt, ist in Form einer einheitlichen baumartigen Dateistruktur organisiert, in der jeder Benutzer über einen eigenen Teil des Dateibaumes (einen Ast) verfügt. Hier darf der Benutzer Dateien erzeugen, löschen und ausführen; dem Superbenutzer `root` gehört der gesamte Baum (vgl. Abbildung A.1). Ob der gesamte Dateibaum auf einer einzigen Platte gespeichert oder auf eine Vielzahl von Platten verteilt ist, ist für den Benutzer weitgehend transparent.

Das UNIX-Dateisystem unterscheidet zwischen *Verzeichnissen* und (anderen) *Dateien*. Die Verzeichnisse enthalten andere Verzeichnisse und Dateien; Dateien können zum Beispiel Texte oder ausführbare Programme enthalten. Alle inneren Knoten des Dateibaumes, die Wurzel `/` eingeschlossen, sind Verzeichnisse. Die Blätter des Baumes sind (noch) leere Verzeichnisse oder andere Dateien. Man beachte: Verzeichnisse sind letztlich auch Dateien, nur mit ganz speziellem Inhalt.

Die Lage einer Datei (oder eines Verzeichnisses) in der Baumstruktur kann durch ihren *Pfad* beschrieben werden, d.h. den Weg von der Wurzel des Baumes zu der betreffenden Datei. Der *Pfadname* besteht aus der Folge der Namen der dabei durchlaufenen Knoten, wobei die einzelnen Dateinamen durch einen Schrägstrich (`/`) voneinander getrennt werden. Die Wurzel wird nur durch einen Schrägstrich bezeichnet.

Der so angegebene Weg von der Wurzel zu einer Datei ist gleichzeitig ihr vollständiger Name. Zum Beispiel sind die vollen Namen der Dateien `prog.c` und `a.out` im Baumabschnitt von `benutzer2` im Beispiel (vgl. Abbildung A.1)

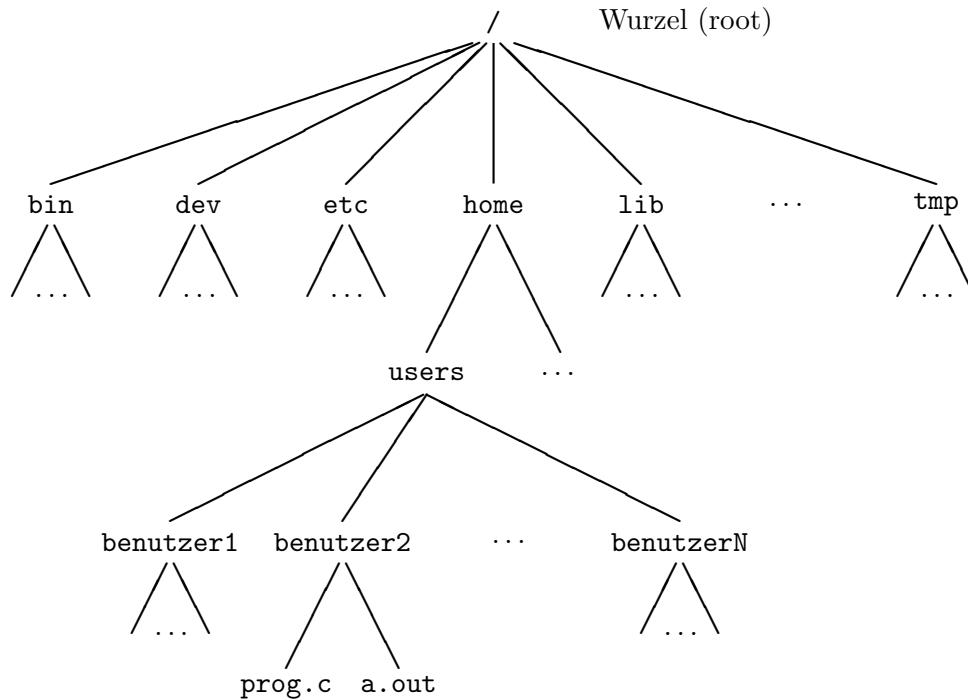


Abbildung A.1: Beispiel eines UNIX-Filesystems

```

/home/users/benutzer2/prog.c
/home/users/benutzer2/a.out

```

Kein Benutzer muss die Namen seiner Dateien stets vollständig angeben, denn das System gibt jedem Dateinamen, der nicht vollständig angegeben wird, ein Präfix, das den Weg von der Wurzel zum aktuellen Baumabschnitt des Benutzers beschreibt. Im Beispiel ist für **benutzer2** das Präfix nach dem Einloggen `/home/users/benutzer2/`. So sind die Eingabe

```
a.out ↵
```

und die Eingabe

```
/home/users/benutzer2/a.out ↵
```

äquivalent. Welches Präfix das System momentan den Dateinamen gibt, lässt sich durch die Eingabe des Kommandos

```
pwd ↵
```

(*print working directory*) feststellen. Das Präfix ist der Pfadname eines Verzeichnisses, das man auch Arbeitsverzeichnis (*working directory*) nennt. Eine Abkürzung dafür ist der Punkt (`.`). So kann man alternativ auch

```
./a.out ↵
```

für obigen Aufruf verwenden. Das kann sogar notwendig sein, wenn nach ausführbaren Programmen nicht im Arbeitsverzeichnis gesucht wird.

Auf eine andere Möglichkeit, ein Präfix vom System automatisch ergänzen zu lassen, wird gleich noch näher eingegangen.

A.5 Dateiverwaltung

Das momentan gültige Namenspräfix kann man durch das Kommando

```
cd pfadname ↵
```

(*change directory*) ändern, wobei *pfadname* den Weg von der Wurzel zu einem Verzeichnis beschreibt. (Bei der Ausführung des Kommandos wird der angegebene Pfadname ggf. noch durch das alte Präfix ergänzt.)

Mit dem Kommando

```
cd .. ↵
```

wird das gegenwärtige Präfix um den letzten Namen gekürzt. Dieses entspricht einer Bewegung in der Richtung zur Wurzel im Dateibaum. Mit dem Kommando

```
cd name ↵
```

wird das gegenwärtige Präfix um den angegebenen Namen erweitert, sofern in der gegenwärtigen Verzeichnisdatei eine Verzeichnisdatei mit diesem Namen vorhanden ist. Dieses entspricht einer Bewegung in der Richtung weg von der Wurzel im Dateibaum.

Nach dem Einloggen befindet man sich in seinem Heimatverzeichnis (*home directory*), d.h. in der Wurzel des eigenen Teilbaumes. Für **benutzer2** heißt das etwa, dass nach dem Einloggen

```
/home/users/benutzer2/
```

das aktuelle Präfix ist. Dieses Präfix kann man jederzeit wiederherstellen, indem man nur

```
cd ↵
```

eintippt. Für das Heimatverzeichnis eines Benutzers gibt es auch noch eine spezielle Kurzform, nämlich

```
~benutzer2/
```

Das System bestimmt dann selbst, wie der Pfad zu diesem Verzeichnis korrekt zu ergänzen ist.

Welche Dateien ein Verzeichnis mit dem Namen *name* enthält, lässt sich mit dem Kommando

```
ls name ↵
```

(*list*) feststellen. Beispiele:

```
ls / ↵  
ls /home ↵  
ls /home/users/benutzer2 ↵  
ls ~benutzer2 ↵
```

Das erste Kommando zeigt alle Einträge in der Wurzel, das zweite alle Einträge im Verzeichnis **/home**, das dritte und vierte alle Einträge im **home**-Verzeichnis des Benutzers **benutzer2**.

Gibt man nur das Kommando

```
ls ↵
```

ein, so werden alle Einträge im aktuellen Verzeichnis angezeigt, d.h. dem Verzeichnis, dessen Pfadname durch **pwd** geliefert wird.

Mit der Option **-l** (*long*)

```
ls -l ↵
```

wird für jeden Eintrag des Verzeichnisses mehr Information ausgegeben: Jede Zeile enthält Angaben zu einer Datei, deren Name am Ende der Zeile steht. Die ersten 10 Zeichen haben die Form

```
d rwx rwx rwx
```

und bedeuten:

- d** es handelt sich um ein Verzeichnis
- r** die Datei darf gelesen werden
- w** in die Datei darf geschrieben werden
- x** die Datei darf (als Programm) ausgeführt oder (als Verzeichnis) aufgelistet werden

Steht an der Position von **d** ein Bindestrich (-), so handelt es sich *nicht* um ein Verzeichnis. An den anderen Stellen bedeutet ein Bindestrich, dass das entsprechende Zugriffsrecht *nicht* besteht. Die drei Gruppen **rwx** markieren, von links nach rechts, die Zugriffsrechte für den Besitzer der Datei, für seine Gruppe und für andere Benutzer.

Die Zugriffsrechte, also der Schutz der Dateien, können mit dem Kommando **chmod** (*change mode*) geändert werden. Sehen Sie sich die Einzelheiten einmal selbst mit dem Kommando **man** an.

Mit der Optionenkombination **-al**

```
ls -al ↵
```

werden auch Dateien erfasst, deren Name mit einem Punkt beginnt und die sonst nicht aufgelistet werden.

Neue Verzeichnisse kann man mit dem Kommando

```
mkdir name ↵
```

(*make directory*) anlegen, leere Verzeichnisse mit dem Kommando

```
rmdir name ↵
```

(*remove directory*) löschen, entsprechende Zugriffsrechte vorausgesetzt. Andere Dateien kann man mit

```
rm name1 name2 ... nameN ↵
```

(*remove*) löschen, ebenfalls entsprechende Zugriffsrechte vorausgesetzt. Beim Löschen ist oft die Option **-i**

```
rm -i name1 name2 ... nameN ↵
```

zweckmäßig. Sie bewirkt, dass man für jede einzelne Datei das Löschen durch Eintippen von **y(es)** noch einmal ausdrücklich bestätigen muss.

Kopieren bzw. Umbenennen kann man Dateien mit den Kommandos

```
cp quelle ziel ↵
```

(*copy*) bzw.

```
mv quelle ziel ↵
```

(*move*). Durch Umbenennen kann eine Datei auch aus einem Verzeichnis in ein anderes verlegt werden.

A.6 Metazeichen

Bei vielen Kommandos ist es zweckmäßig, Dateinamen nicht explizit anzugeben, sondern die *Metazeichen* (*Jokerzeichen*, *wild cards*) Fragezeichen (?) und Stern (*) zu verwenden:

- Ein Fragezeichen steht für genau ein beliebiges Zeichen.
- Ein Stern steht für eine beliebig lange (auch leere) Folge beliebiger Zeichen.

Beispiele:

```
rm *.c ↵
rm *~ ↵
rm * ↵
rm -i * ↵
rm aufg?.c ↵
```

Setzen wir voraus, dass die entsprechenden Zugriffsrechte bestehen, so gilt: Im ersten Beispiel werden in der aktuellen Directory alle Dateien mit dem Suffix `.c` gelöscht und im zweiten Beispiel alle Dateien, bei denen das letzte Zeichen des Namen `~` ist; im dritten Beispiel werden in der aktuellen Directory **alle** Dateien ausnahmslos und ohne Rückfrage gelöscht; im vierten Beispiel werden in der aktuellen Directory alle Dateien gelöscht, für die dieses ausdrücklich bestätigt wird; im letzten Beispiel werden in der aktuellen Directory alle Dateien gelöscht, deren Name mit `aufg` beginnt, dann ein beliebiges Zeichen aufweist und danach mit `.c` endet.

A.7 Auflisten von Datei-Inhalten

Durch

```
cat datei ↵
```

wird der Inhalt der entsprechenden Datei aufgelistet. Dieses Kommando empfiehlt sich allerdings nur für ziemlich kurze Dateien, da die Ausgabe „durchrollt“, wenn sie länger als eine Bildschirmseite ist. Seitenweise kann man sich den Inhalt einer Datei mit dem Kommando

```
more datei ↵
```

ansehen: Die Ausgabe stoppt, sobald der Bildschirm voll ist; durch Drücken der Leertaste wird die nächste Bildschirmseite gezeigt, durch Drücken der ENTER-Taste nur die nächste Zeile.

Zur Ausgabe auf einen Drucker dient das Kommando

```
lpr datei ↵
```

(*lineprinter*) oder, wenn verschiedene Drucker zur Verfügung stehen

```
lpr -Pxyz datei ↵
```

wobei `xyz` für den Namen des Druckers steht. Leider kommt es immer wieder vor, dass Drucker durch Fehlbedienung blockiert werden. Den Status eines Druckers kann man sich mit dem Kommando

```
lpq -Pxyz ↵
```

(*lineprinter queue*) ansehen. Stellt man fest, dass die Warteschlange des Druckers nicht leer ist, der Drucker aber trotzdem nicht arbeitet, kann es daran liegen, dass er blockiert ist. Falls eine eigene Datei der „Übeltäter“ ist, kann man versuchen, die Datei aus der Warteschlange zu löschen. Hierzu dient das Kommando

```
lprm -Pxyz # ↵
```

(*lineprinter remove*), bei dem man als Nummer die Zahl einzusetzen hat, die das Kommando `lpq` für die Datei gemeldet hat.

A.8 Umleitung von Ein- und Ausgabe

UNIX-Kommandos holen ihre Eingabe vielfach vom *Standardeingabegerät* (Tastatur) und schreiben ihre Ausgabe vielfach auf das *Standardausgabegerät* (Bildschirm).

Die Standardein-/ausgabe lässt sich aber beim Aufruf eines Kommandos ohne weiteres umleiten.

```
programm < name
```

bewirkt, dass die Datei (oder das Gerät) mit dem Namen *name* als Standardeingabegerät verwendet wird;

```
programm > name  
programm >> name
```

bewirken, dass die Datei (oder das Gerät) mit dem Namen *name* als Standardausgabegerät verwendet wird.

Beide Formen unterscheiden sich nur dann, wenn die Datei, in die die Ausgabe umgeleitet wird, bereits existiert: Bei einem Winkel wird der bisherige Inhalt der Datei zunächst gelöscht; bei zwei Winkeln wird die Ausgabe an den bisherigen Inhalt der Datei angehängt.

Umleitung von Ein- und Ausgabe ist bei vielen Kommandos möglich. Besonders interessant ist die Umleitung für eigene Programme. Man kann für umfangreichere Tests die benötigte Eingabe in einer Datei speichern und diese als Eingabequelle verwenden. So muss die Eingabe nicht jedesmal von Hand erneut eingegeben werden.

A.9 Editieren

Meist hat man die Wahl zwischen verschiedene *Editoren*, d.h. Programmen, mit denen man Texte, zum Beispiel auch C-Quellprogramme eingeben kann. Wichtig ist, dass der Editor den eingegebenen Quelltext ohne Formatierungsanweisungen oder Anweisungen zur Dokumentorganisation speichert. Ein normales Textverarbeitungsprogramm, mit dem man z.B. seine Briefe verfasst ist daher in der Regel nicht geeignet. Mit der Zeit entwickelt sich jeder eine Vorliebe für einen bestimmten Editor.

Es würde wenig bringen, an dieser Stelle zu versuchen, die Funktionsweise eines Editors zu erklären oder auf spezielle Funktionen eines bestimmten Editors einzugehen. Neben den oft vorinstallierten Editoren gibt es eine Reihe freier Editoren, die man kostenlos im Internet herunterladen kann. Es lohnt sich auf jeden Fall zunächst einige verschiedene Editoren auszuprobieren und sich erst dann zu entscheiden.

A.10 Übersetzen und Binden

Prinzipiell sind zwei Schritte erforderlich sind, um aus dem Quellcode eines C-Programms ein ausführbares Programm zu erzeugen:

1. Der Quellcode wird mit dem *Compiler* übersetzt. Der Compiler erzeugt eine *Objektdatei*.
2. In der Regel werden Bibliotheksroutinen benötigt. Die Objektdatei und diese Routinen werden durch den *Linker* gebunden.

Dazu benötigt man einen Compiler und Linker. Auf Linux-basierten Systemen ist meist die GNU Compiler-Sammlung (GNU Compiler Collection, GCC, <http://gcc.gnu.org>) zusammen mit der GNU C Bibliothek (GNU C Library, glibc, <http://www.gnu.org/software/libc>) installiert. Beide kann man kostenlos im Internet herunterladen.

Die beide Schritte – compilieren und linken – werden vom `gcc` mit einem einzigen Kommando bewirkt:

```
gcc prog.c
```

Dabei ist `prog.c` der Name der C-Quelldatei; das Suffix `.c` zeigt dem Compiler an dass es sich um C-Quellcode handelt. Das ausführbare Programm erhält den Namen `a.out`. Die Eingabe dieses Namens als UNIX-Kommando bewirkt die Ausführung des Programms.

Soll das ausführbare Programm einen anderen Namen als `a.out` erhalten, zum Beispiel `prog`, so muss man die Option `-o` und den neuen Namen zusätzlich in das `gcc`-Kommando eintragen:

```
gcc -o prog prog.c
```

Eine zusätzliche Option sollte man beim Aufruf des `gcc` auf jeden Fall angeben, nämlich `-Wall`, so dass der Aufruf

```
gcc -Wall [-o prog] prog.c
```

ist. Ist diese Option angegeben, schreibt der Compiler zu „verdächtigem“ Code, den er sonst kommentarlos akzeptieren würde, Warnungen. In vielen Fällen weisen diese Warnungen auf Programmierfehler hin, ohne deren Beseitigung das Programm nicht laufen wird. Merke: Ein Programm, das ein Compiler widerspruchlos akzeptiert, ist noch lange kein korrektes Programm!

Die Option `-O` optimiert den erzeugten Maschinencode hinsichtlich Größe und Geschwindigkeit. Die Ausführung eines Programms kann so deutlich beschleunigt werden. Der Compiler kann aber auch mehr Warnungen ausgeben. Durch die bei der Optimierung verwendete Datenfluss-Analyse wird zum Beispiel auch erkannt, ob eine Variable ggf. uninitialized verwendet wird.

Zwei weitere Optionen, nämlich `-ansi` und `-pedantic`, unterstützen den Programmierer, der sauberes Standard-C programmieren und keine Erweiterungen des `gcc` verwenden möchte.

Der normale einfache Compileraufruf sieht also so aus:

```
gcc -Wall -ansi -pedantic -O [-o prog] prog.c
```

Eine weitere oft benötigte Option ist `-lm`. Sie sorgt dafür, dass der Bibliotheksteil mit den mathematischen Funktionen gebunden wird. Da es sich um eine Linker-Option handelt, wird sie nach den Quelltextdateien angegeben.